

Infrastructure as Code



BYTEBYTEGO

SEP 12, 2024 • PAID



238



1



21

Share

It is difficult to imagine modern businesses operating without a scalable process to provision and manage the infrastructure.

Scalable infrastructure management is critical for businesses to adapt quickly to fluctuating workloads and user demands without compromising performance or incurring excessive costs. It offers several key benefits from multiple perspectives:

- **Availability:** Scalable infrastructure management ensures that the application remains up and running at all times. It minimizes downtime and enables continuous service availability by dynamically adjusting resources based on demand.
- **Application Performance:** Scalable infrastructure ensures that application performance is not affected by changes in workload. It allows for the seamless addition or removal of resources to maintain optimal performance levels, even during peak periods.
- **Better Quality of Service:** By enhancing availability and application performance, scalable infrastructure management allows businesses to offer better service quality to their customers. This improved service quality can lead to higher customer satisfaction and loyalty.
- **Cost Efficiency:** Scalable infrastructure management provides opportunities for cost optimization by scaling down resources during idle times.

Modern cloud-based infrastructure is particularly adept at scaling to support millions of users.

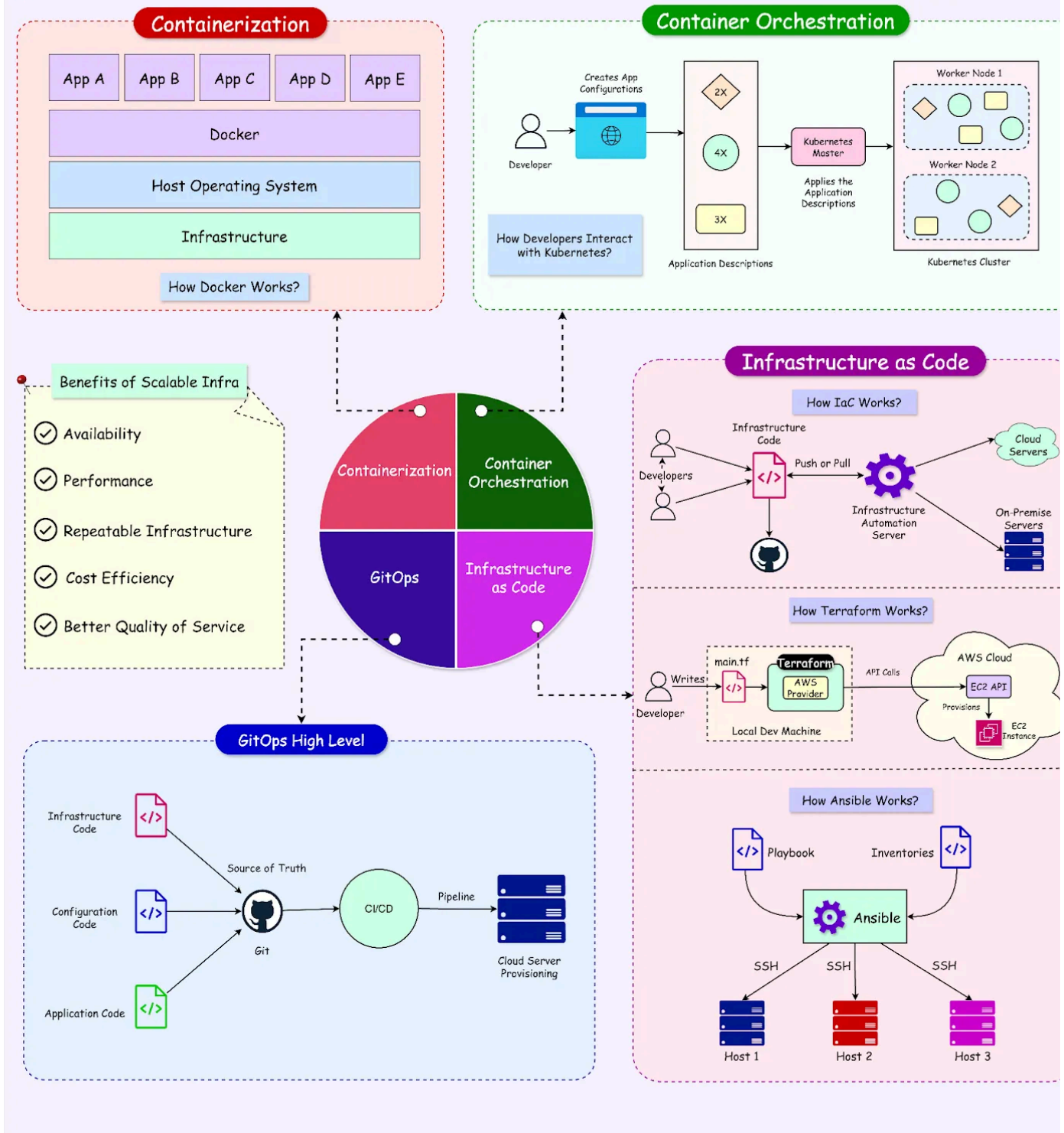
However, while it may seem effortless, managing scalability beyond simple autoscaling groups and load balancers can become complex.

As the codebase grows due to the involvement of numerous engineers, the potential for mistakes also increases. While minor issues like syntax errors or forgotten comments can be mitigated quickly, more serious mistakes such as leaked security keys, improper storage security settings, or open security groups can have disastrous consequences.

Therefore, it is crucial to formulate effective ways to manage and provision the infrastructure.

In this post, we'll explore the most important ways of scaling the provisioning and management of modern application infrastructure.

A Cheatsheet on Scaling Infrastructure Provisioning and Management



What is Non-Scalable Infrastructure?

Traditionally, infrastructure provisioning and management has been a non-scalable activity.

Developers often rely on using special scripts and privileges on the server machines to manage the infrastructure of their projects. Also, some non-scalable strategies, such as using git repositories and basic security controls, are essential at any scale to prevent data loss and ensure code integrity.

However, while some non-scalable methods of managing infrastructure remain popular and useful for single-developer shops and small stacks, the majority are no longer suitable for large-scale operations at the enterprise level.

These methods often lack the standardization abstraction required for infrastructure management and provisioning at scale. Some examples are as follows:

- When building infrastructure, the key is to write readable and maintainable code. While coding from terminals is acceptable and necessary for developer freedom and consistency, and security can be compromised when developers run infrastructure deployments directly from their terminals.
- Developers need the freedom to manage their environment completely for experimentation and iteration, but this freedom should be limited at the development stage.
- Terminals should only have access to the repository where the code is committed and not to any further resources. Handing out access keys to cloud environments can create significant security risks, especially when deploying to multiple environments.

Let us now look at some of the most popular strategies to make it easier to scale infrastructure management. Note that many of these strategies can and should be used in conjunction as the application's requirements evolve.

Containerization

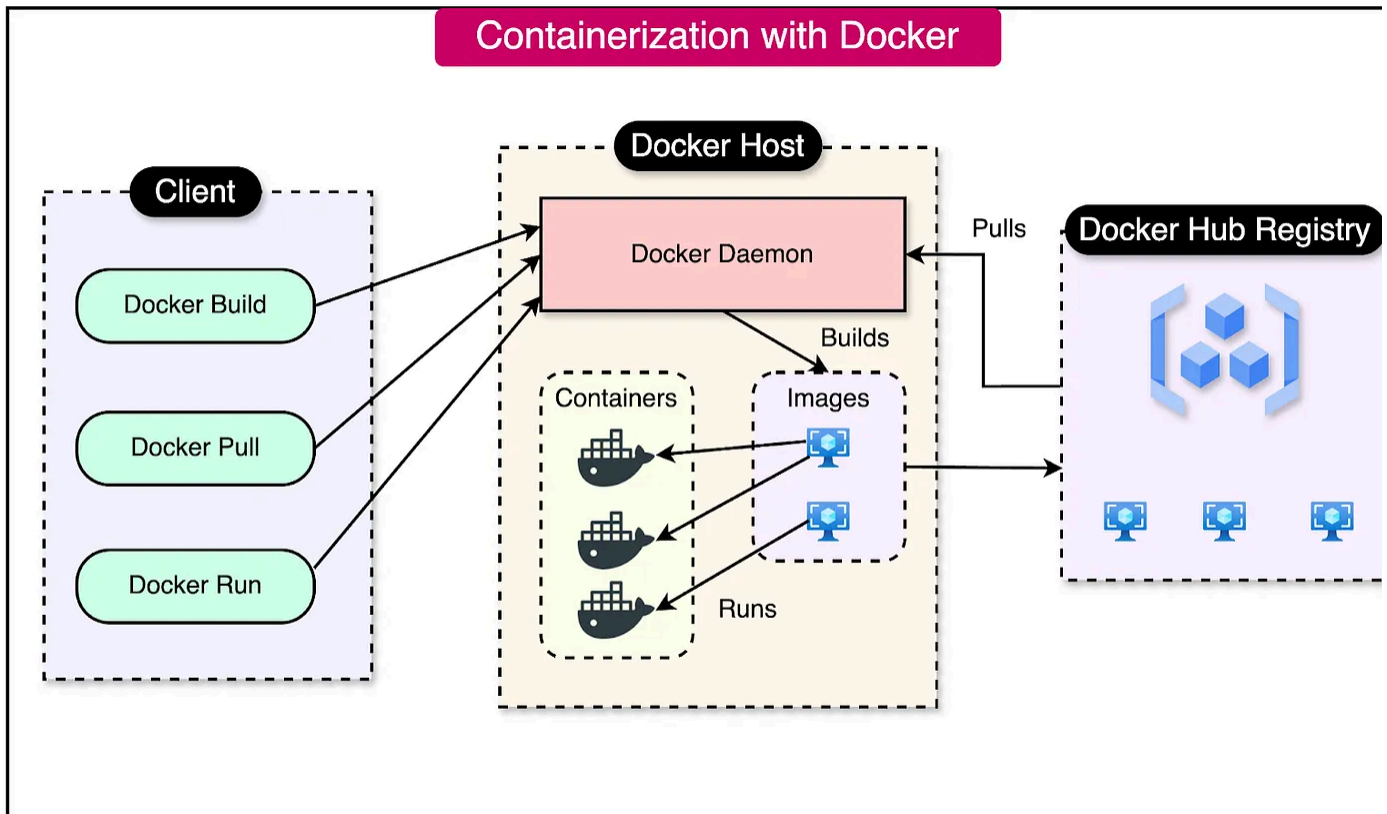
Containerization is a powerful approach for scaling the infrastructure layer of an application. It is a lightweight form of virtualization that packages an application and its dependencies into a standardized unit called a container.

Docker is the most popular containerization platform, widely adopted for its simplicity and efficiency.

Some key concepts to understand Docker are as follows:

- **Container:** A container is a standalone, executable package that includes everything needed to run a piece of software. This includes the application code, runtime, libraries, and dependencies.
- **Image:** An image serves as a template for creating containers. It contains the application code, runtime, libraries, and dependencies necessary for the application to run.
- **Dockerfile:** A Dockerfile is a script that defines how to build a Docker image. It specifies the base image, copies files, installs dependencies, sets environment variables and defines the command to run the application.
- **Docker Host:** A Docker host is a machine (physical or virtual) that runs the Docker daemon and can execute the Docker containers. The daemon is a background service that manages Docker objects such as images, containers, networks, and volumes.
- **Docker Hub:** Docker Hub is a cloud-based registry for storing and sharing Docker images. It provides a centralized location for managing and distributing images, making collaboration and deployment of applications easy.

The diagram below shows the high-level picture of containerization with Docker.



Using Docker

Here is a basic Dockerfile for a Node.js web application:

```
# Use an official Node.js runtime as the base image
FROM node:14

# Set the working directory in the container
WORKDIR /usr/src/app

# Copy package.json and package-lock.json
COPY package*.json ./

# Install application dependencies
RUN npm install

# Copy the rest of the application code
COPY . .
```

```
# Expose the port the app runs on
EXPOSE 3000

# Define the command to run the app
CMD ["node", "app.js"]
```

The Docker image for this file can be built and run using the below commands:

```
docker build -t my-nodejs-app
docker run -p 3000:3000 my-nodejs-app
```

Scaling With Docker

Using this Dockerfile, we can easily scale the application:

For example, running multiple containers of the same image is a matter of executing the below commands. Each container is like an instance of the application.

```
docker run -d -p 3001:3000 my-nodejs-app
docker run -d -p 3002:3000 my-nodejs-app
```

A reverse proxy like Nginx can be used to distribute traffic among these instances. Also, it becomes easy to update the application by building a new image version and replacing the running containers.

This Dockerfile provides a foundation for containerizing a web application, making it easier to deploy, scale, and manage in various environments. The same principles can be applied to other types of applications by adjusting the base image and the image build steps accordingly.

Container Orchestration

Once the application infrastructure is spread across multiple containers, it becomes tough to manage the lifecycle of all these containers. This is where container orchestration tools become important.

Kubernetes is one of the most popular container orchestration platforms in the world. It provides several features that significantly contribute to the scalability of the infrastructure provisioning, making it an ideal choice for modern application development.

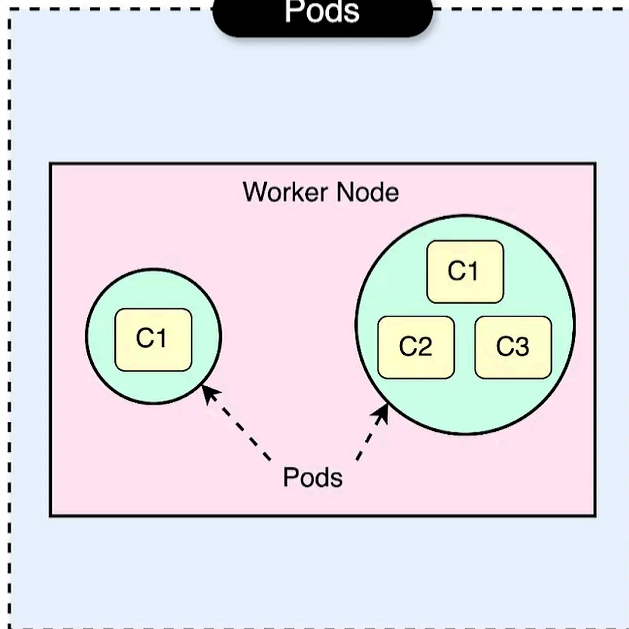
Some essential Kubernetes concepts are as follows:

- **Pod:** The smallest deployable unit in Kubernetes, typically containing one or more containers.
- **Deployment:** Manages a set of identical pods, allowing for easy scaling and updates.
- **Service:** An abstraction layer that defines a logical set of pods and a policy to access them.
- **Volume:** An abstraction that provides persistent storage to pods, allowing data to persist beyond the lifetime of individual containers.
- **Ingress:** Manages external access to services in a cluster.
- **ConfigMap and Secret:** Used for configuration management.

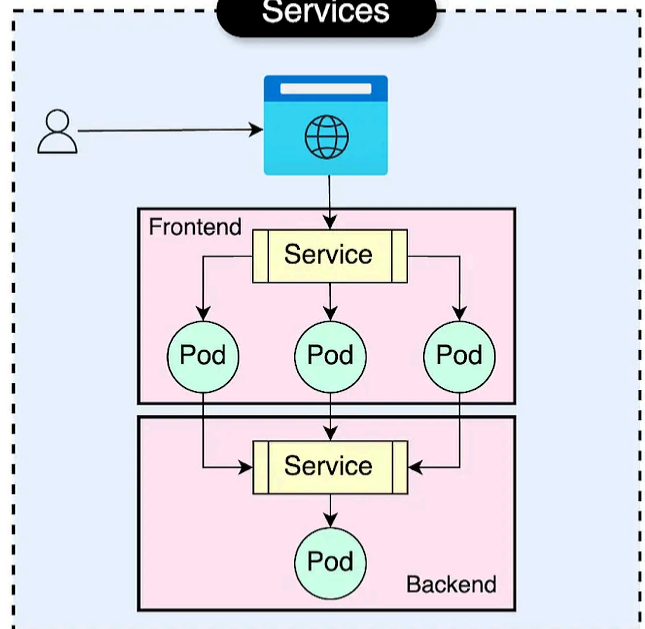
Here's a quick look at some of the concepts and resources that Kubernetes provides

Key Kubernetes Concepts

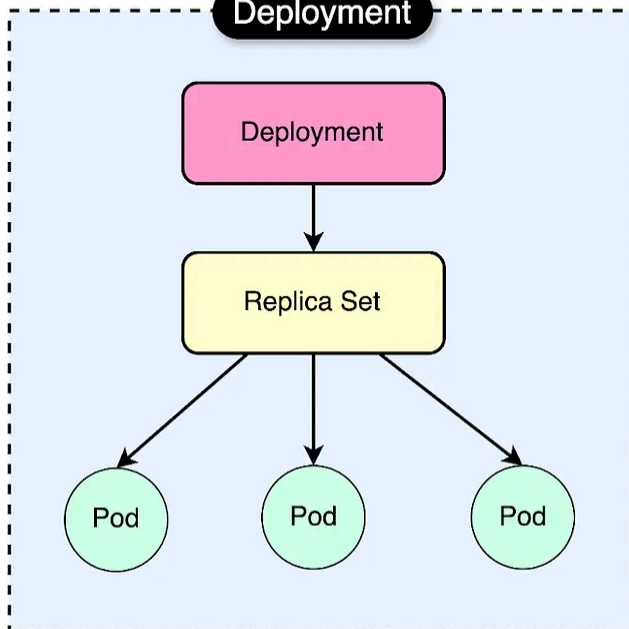
Pods



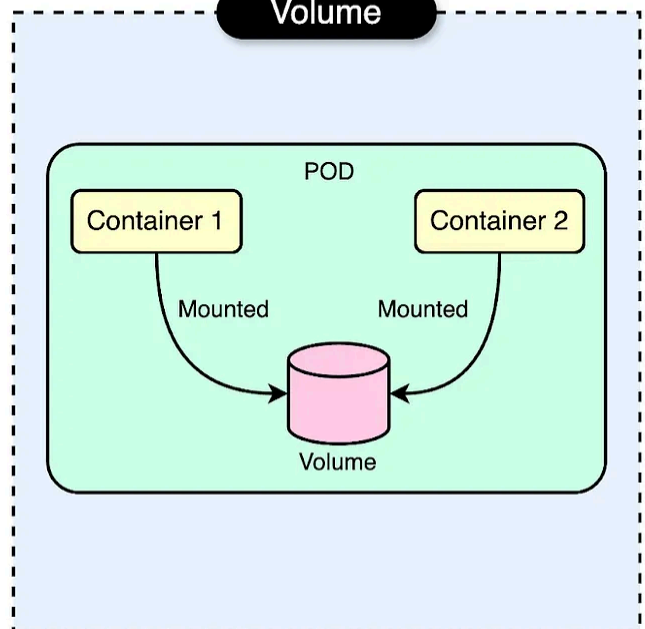
Services



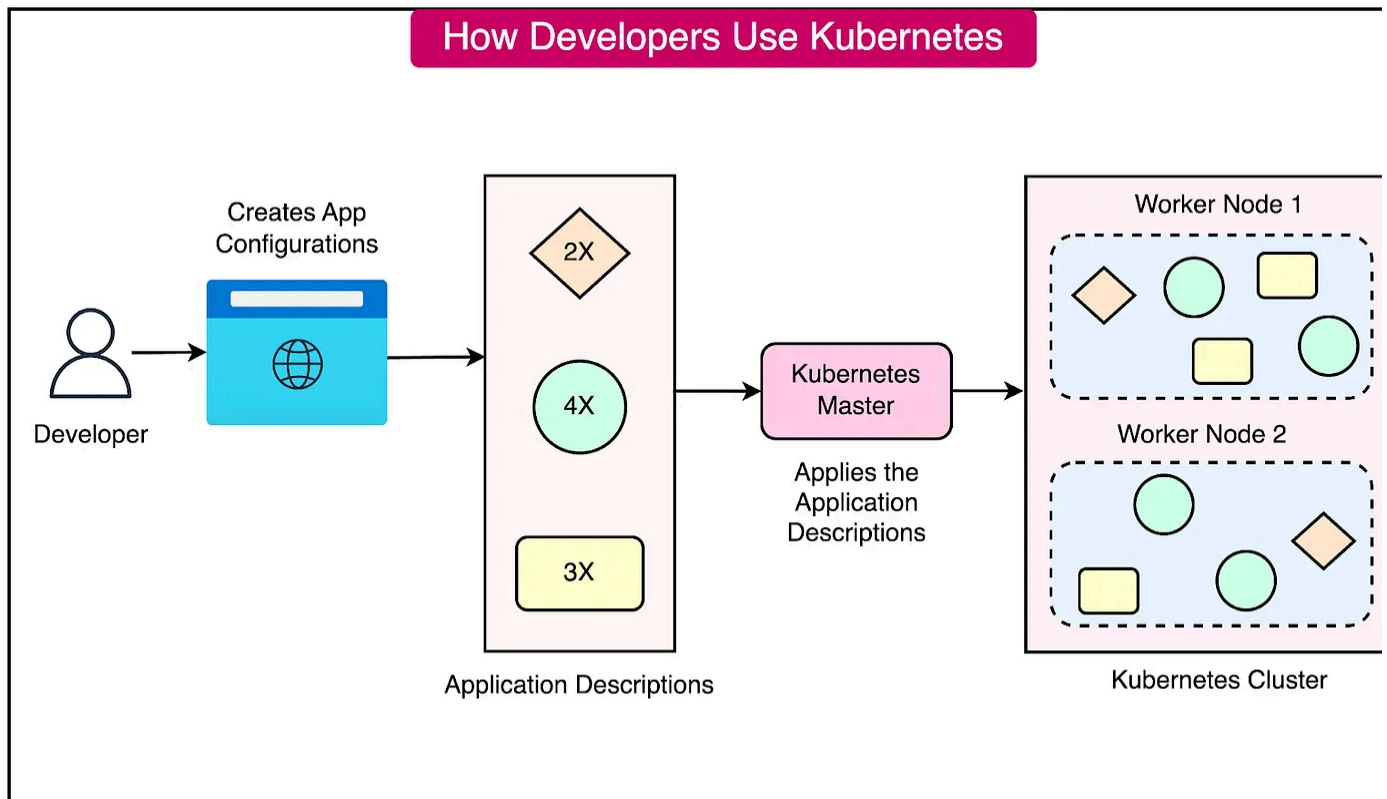
Deployment



Volume



The diagram below shows how developers interact with Kubernetes and what happens behind the scenes.



Scaling the Infrastructure With Kubernetes

Kubernetes can automatically scale the number of pods based on CPU utilization or custom metrics.

Here's an example configuration code for a Horizontal Pod Autoscaler.

```
apiVersion: autoscaling/v2beta1
kind: HorizontalPodAutoscaler
metadata:
  name: myapp-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: myapp
  minReplicas: 2
  maxReplicas: 10
```

```
metrics:
- type: Resource
  resource:
    name: cpu
    targetAverageUtilization: 50
```

On a high level, this HPA configuration ensures that the number of replicas of the myapp deployment is maintained between 2 and 10, scaling up when CPU utilization exceeds 50% and scaling down when it drops below 50%.

Next, Kubernetes Services automatically provides load balancing for a set of pods. See the example below:

```
apiVersion: v1
kind: Service
metadata:
  name: myapp-service
spec:
  selector:
    app: myapp
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8080
  type: LoadBalancer
```

This Service configuration distributes traffic across all pods labeled with app: myapp listening on port 80, and forwarding traffic to port 8080 on the pods.

Kubernetes allows for zero-downtime updates by gradually replacing old pods with new ones. This configuration ensures smooth transitions between version releases.

Here is an example Deployment YAML with a rolling update strategy:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp
spec:
  replicas: 4
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
    spec:
      containers:
      - name: myapp
        image: myapp:v2
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 1
```

Kubernetes automatically restarts failed containers and replaces unresponsive pods. This self-healing behavior is built into Kubernetes and does not require additional configuration. If a pod crashes or becomes unresponsive, Kubernetes will automatically create a new one to replace it based on the desired infrastructure. By using liveness and readiness probes, this restart behavior can be made even more reliable.

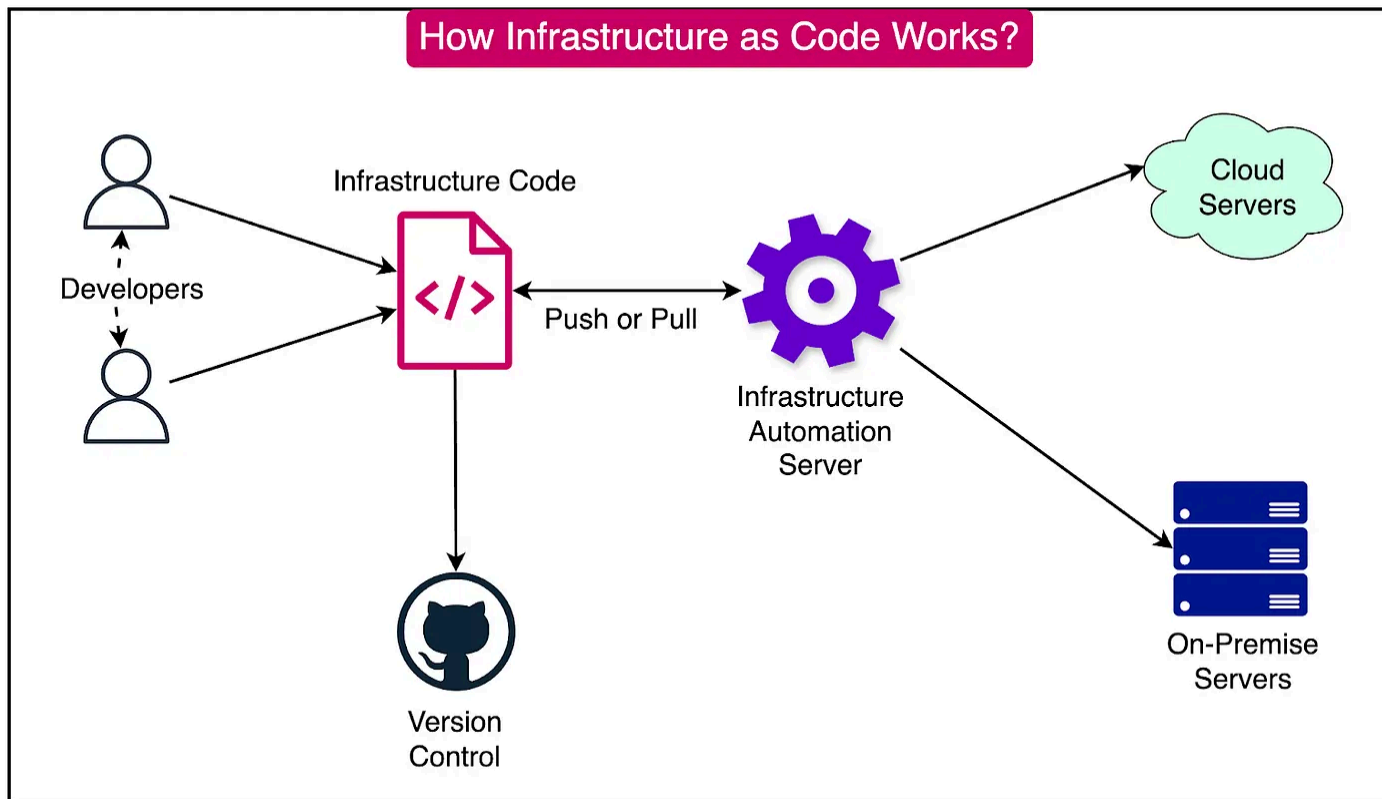
The various features of Kubernetes contribute significantly to the scalability and reliability of applications. Here are some key benefits:

- **Efficient Resource Utilization:** Autoscaling ensures that resources are used efficiently, scaling up or down based on actual demand.
- **High Availability:** Load balancing and self-healing features ensure the application runs even if individual components fail.
- **Consistent Deployments:** Rolling updates ensure smooth transitions between versions, minimizing downtime and disruptions.
- **Adaptability:** The infrastructure can automatically adapt to changing load patterns, ensuring optimal performance and resource utilization.

Infrastructure as Code

Infrastructure as Code (IaC) is a powerful approach that enables the management and provisioning of infrastructure using code. This method treats infrastructure configuration as code, allowing developers to define the infrastructure in files that can be versioned, tested, and reused.

The diagram below shows how infrastructure as code works on a high level.



IaC provides several key benefits:

- **Consistency:** IaC ensures that infrastructure is provisioned consistently across different environments. By defining the infrastructure configuration in code, developers can guarantee that the same setup is applied uniformly, reducing variability and potential errors.
- **Repeatability:** With IaC, developers can easily replicate infrastructure setups. The code-based definition of infrastructure allows for the creation of identical environments, whether for development, testing, staging, or production. This repeatability is crucial for maintaining a consistent and reliable infrastructure.
- **Automation:** IaC reduces manual configuration and errors by automating the provisioning process. By using code to define the infrastructure, developers can leverage tools and scripts to deploy and manage the infrastructure, minimizing the risk of human error.

- **Version Control:** IaC enables tracking changes and rolling back to previous configurations. By storing infrastructure definitions in version control systems like Git, developers can maintain a history of changes, collaborate with team members, and revert to previous versions if needed. This version control capability enhances the reliability and maintainability of the infrastructure.

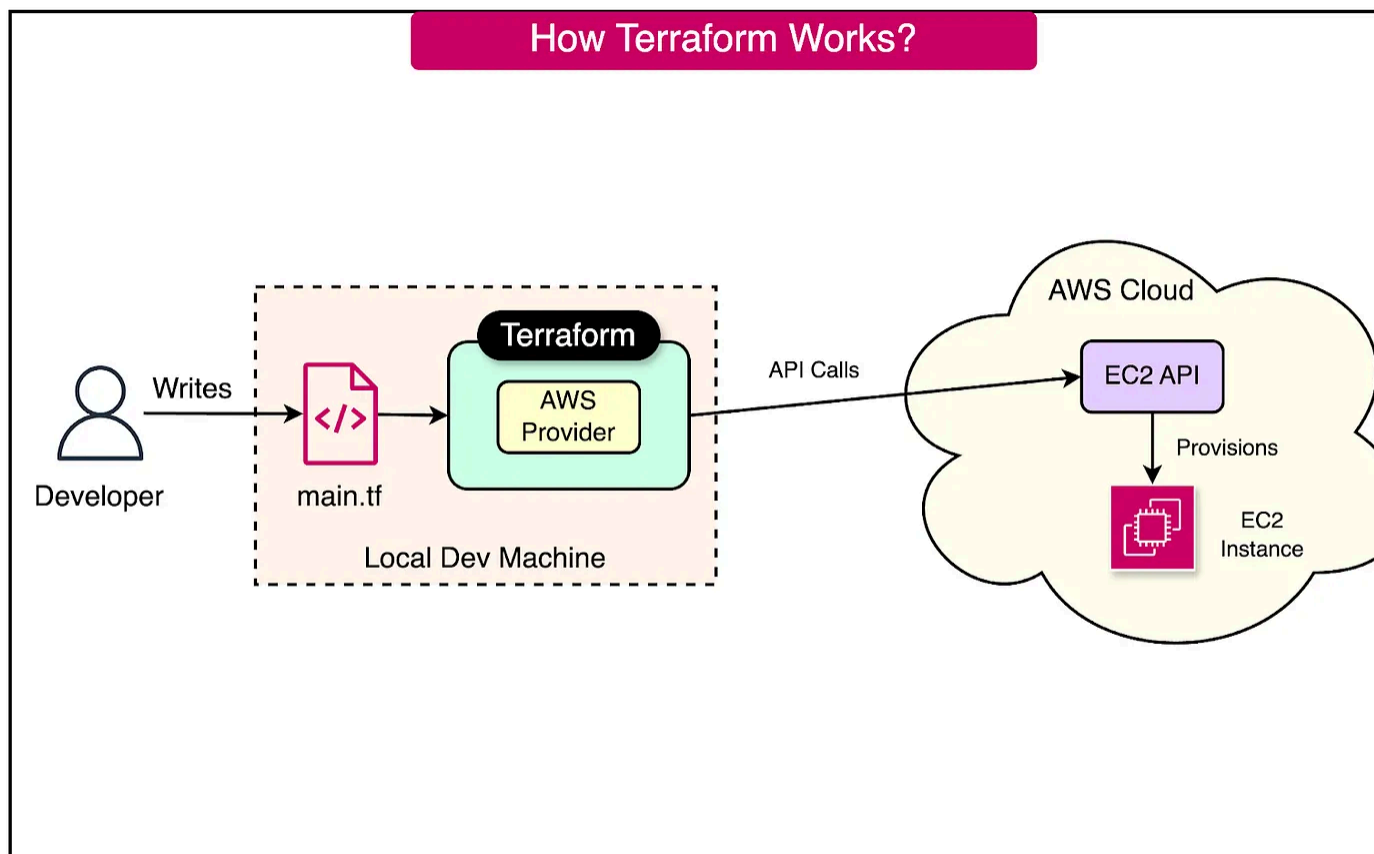
Several IaC tools are available, which can help scale the infrastructure provisioning activities.

Let's look at a few of them in more detail:

1 - Terraform

Terraform is a popular (formerly open-source) tool that enables the definition and provisioning of infrastructure using a high-level configuration language. This tool is particularly useful for managing and scaling infrastructure components efficiently.

See the diagram below that explains how Terraform works to provision an AWS EC instance.



Here's an example of a configuration file that developers can write to provision an AWS EC2 instance in a particular region:

```
provider "aws" {
  region = "us-west-2"
}

resource "aws_instance" "example" {
  ami          = "ami-0c55b159cbfaffe1f0"
  instance_type = "t2.micro"

  tags = {
    Name = "ExampleInstance"
  }
}
```


Terraform offers several features that make it an ideal tool for scaling infrastructure:

- **Easily Create Multiple Instances:** By adjusting the configuration, developers can easily create multiple instances of the same resource. For example, it's possible to modify the `count` attribute in the resource block to specify the number of instances to create.
- **Use Modules for Reusable Configurations:** Terraform modules allow developers to define reusable configurations for scaling infrastructure components. Modules encapsulate related resources and variables, making it easier to manage complex infrastructure setups and scale them as needed.
- **Automate Scaling Processes:** Terraform can be integrated with Continuous Integration/Continuous Deployment (CI/CD) pipelines to automate scaling processes based on predefined triggers and conditions.

2 - AWS CloudFormation

AWS CloudFormation is a service that provides a common language for describing and provisioning AWS infrastructure resources. This service enables developers to define complex AWS architectures as templates, ensuring consistent and repeatable deployments across different environments.

To illustrate how AWS CloudFormation works, let's consider a basic example of creating an S3 bucket:

```
Resources:
  MyS3Bucket:
    Type: "AWS::S3::Bucket"
    Properties:
      BucketName: "my-example-bucket"
```

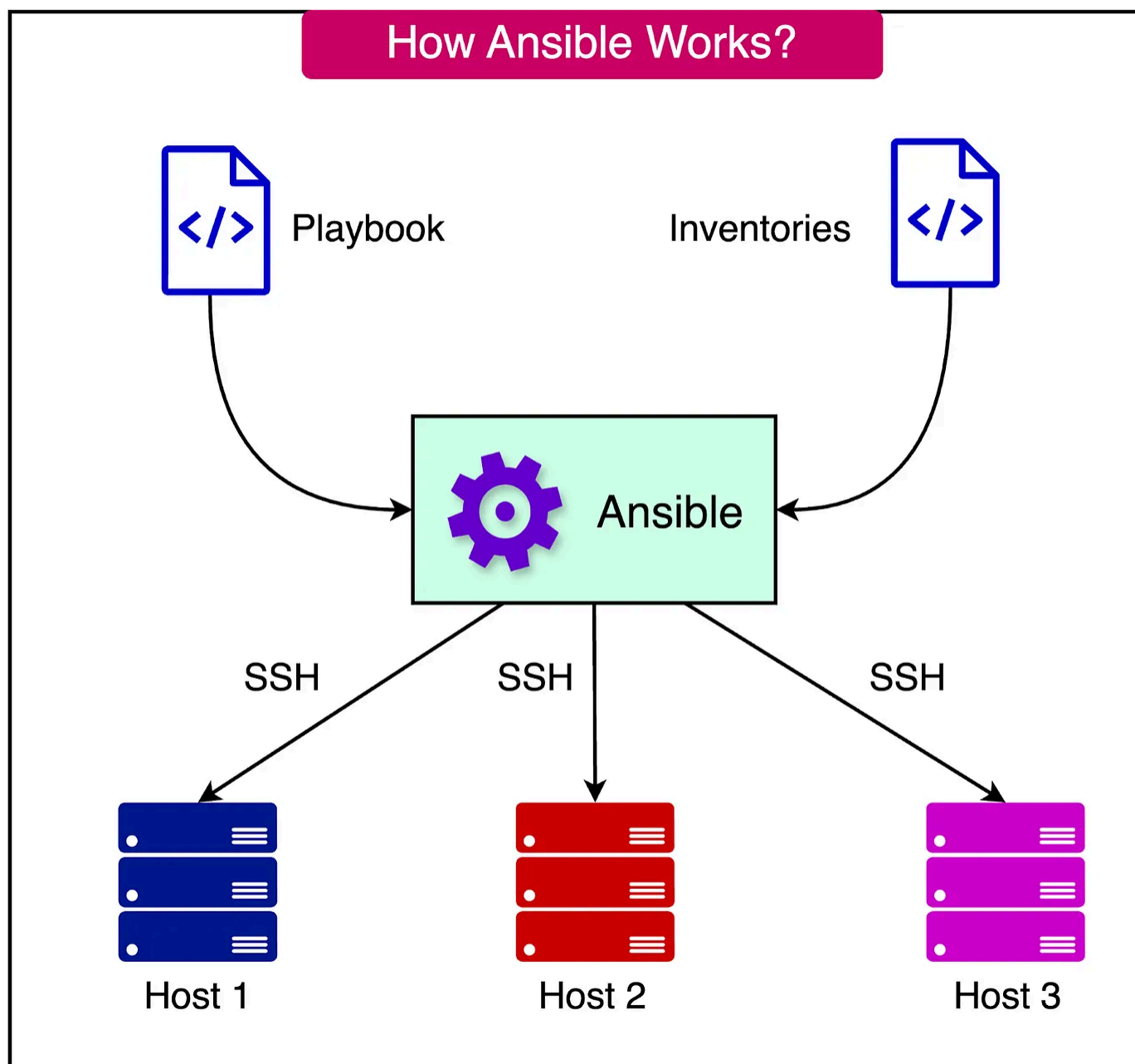
AWS CloudFormation offers several features that make it an ideal tool for scaling AWS infrastructure:

- AWS CloudFormation allows developers to define complex AWS architectures templates. These templates can be deployed consistently across different environments.
- StackSets can be used to manage resources across multiple AWS accounts and regions.
- AWS CloudFormation can be integrated with AWS Auto Scaling services to automate the scaling of AWS resources.

3 - Ansible

Ansible is an open-source automation tool that can be used for configuration management, application deployment, and task automation. It plays a crucial role in ensuring consistency and scalability across multiple servers.

See the diagram below to understand how Ansible works on a high level:



Here's an example code for installing NGINX on a server:

```
- hosts: webservers
  become: yes
  tasks:
    - name: Ensure NGINX is installed
      apt:
```

```
name: nginx
state: present
```

Ansible offers several features that make it an ideal tool for scaling infrastructure:

- **Automate Configuration:** Ansible automates the configuration of multiple servers, ensuring consistency across all nodes. By defining playbooks that specify the desired configuration, we can ensure that all servers are set up uniformly, reducing the risk of errors and inconsistencies.
- **Easy Scaling:** Ansible makes it easy to scale out by adding new servers to the inventory and applying the same playbooks. This approach allows dynamic addition or removal of servers without manual configuration.
- **Reusable Configurations with Roles:** Ansible roles enable developers to define reusable configurations for scaling infrastructure components.

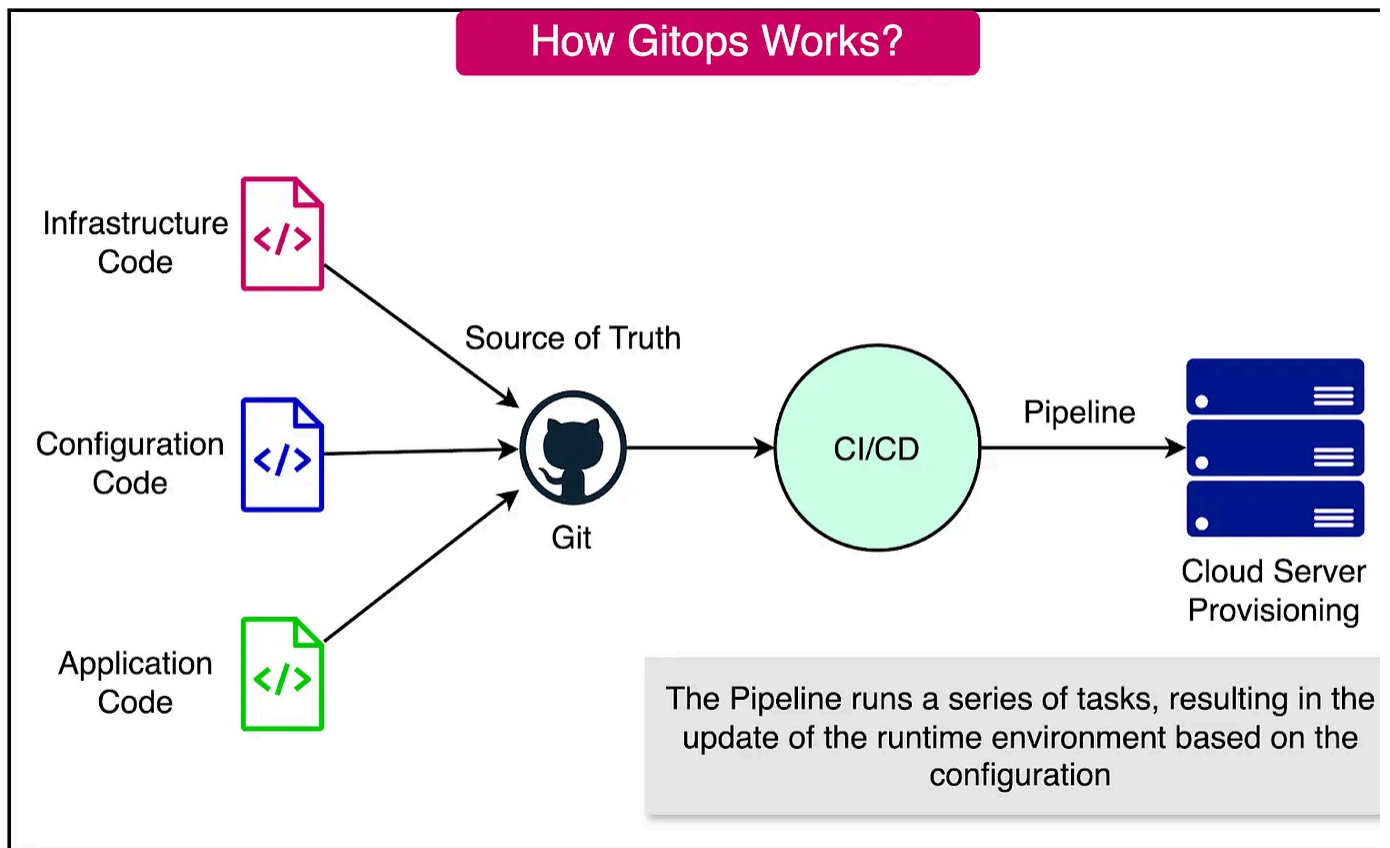
Gitops

It is crucial to ensure that developers have the freedom to control their environment to maximize their productivity. However, this freedom should not come at the cost of collaboration and visibility.

To break down silos and ensure seamless collaboration, GitOps has emerged as a powerful solution.

GitOps leverages a Git workflow combined with continuous integration and continuous delivery (CI/CD) to automate infrastructure updates. This approach treats the Git repository as the single source of truth for all code and the starting point for deployments.

See the diagram below to understand the GitOps approach:



Some key concepts of GitOps are as follows:

- **Single Source of Truth:** The Git repository serves as the central hub for all code ensuring everyone works from the same version of the truth. This centralization helps in breaking down silos by providing a unified view of the codebase.
- **Automated Deployments:** Whenever new code is merged into the repository, the CI/CD pipeline automatically implements the changes in the environment. This automation ensures that updates are consistently applied across all environments, reducing manual errors and inconsistencies.
- **Centralized Deployment Mechanisms:** By centralizing the deployment mechanisms within the Git repository, GitOps makes it easier to ensure that everyone can deploy changes when needed, with the right controls and policies in place.

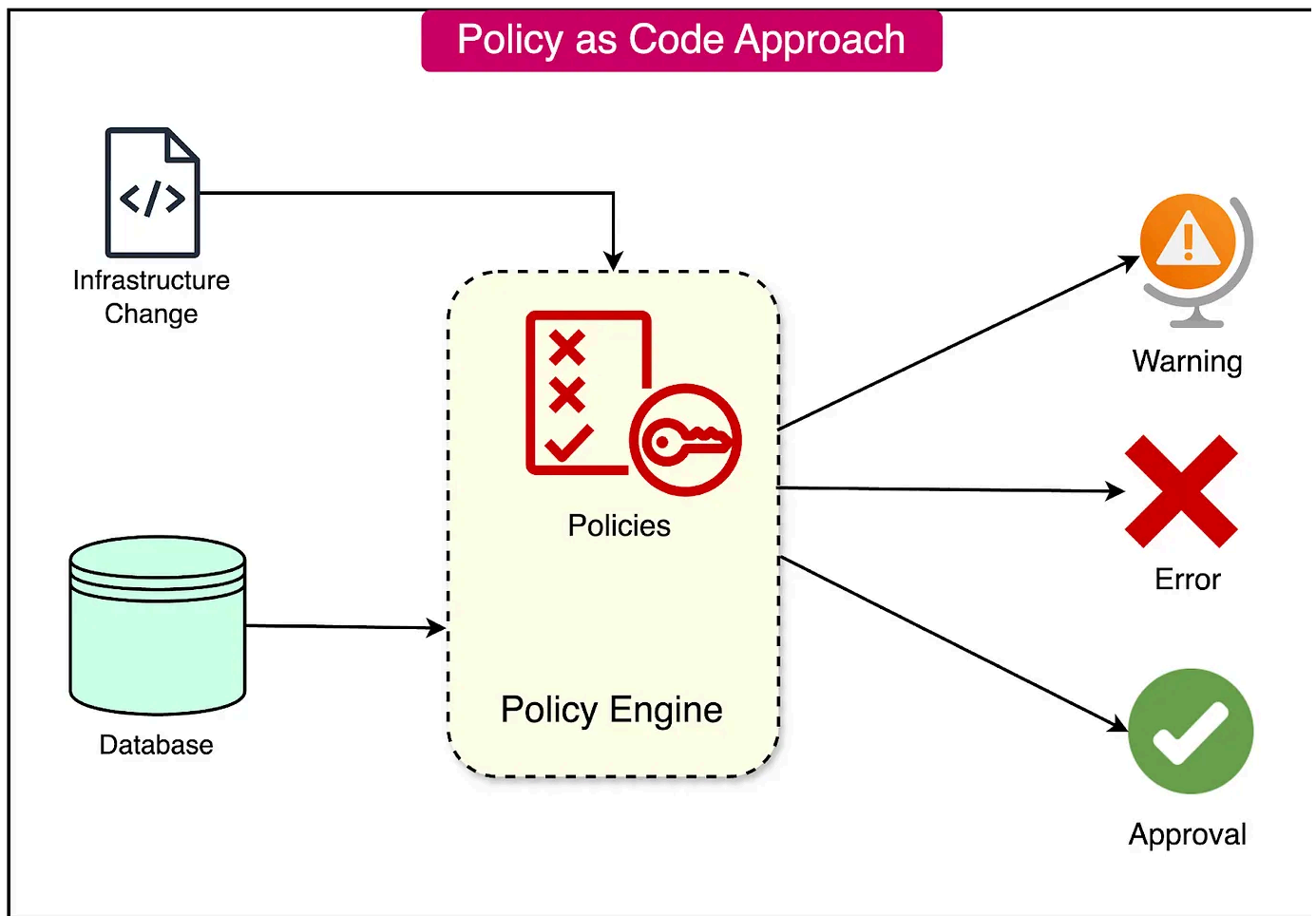
- **Visibility and Transparency:** Requiring all deployed code to be checked into the repository removes silos and provides visibility into what developers are working on.

Use Policy-as-Code and Scoring Algorithms

When scaling infrastructure, it is crucial to ensure that the minimum number of resources are affected in case something goes wrong. Managing thousands of resources can be complex, and a single configuration disaster can lead to prolonged recovery times, often taking hours or even days.

To mitigate such risks, policy-as-code tools like Open Policy Agent (OPA) and clever scoring algorithms can be employed to score resources based on their importance.

The diagram below shows a typical workflow for implementing a policy-as-code approach.



OPA allows developers to define policies using its scripting language, Rego, which can be used to score resources based on their criticality.

Consider an example where resources are scored based on their importance:

- EC2 Instance in a 100 EC2 Instance Autoscaling Pool: 1 point
- Redundant Load Balancer in a Dual LB Setup: 25 points
- Production Database: 100 points

Using OPA's Rego language, developers can set a threshold to require manual intervention if the total points exceed a certain value. For example, setting the threshold at a value of 49 points (depending on the point distribution) can ensure that critical resources are not modified without proper authorization.

Using Immutable Infrastructure

When developing and updating infrastructure, there are two primary methods to consider: mutable and immutable infrastructure.

Traditionally, developers have relied on mutable infrastructure, where updates and modifications are made directly to the live infrastructure. This approach allows for easy adjustments as needed, but it can lead to configuration drift.

Configuration drift occurs when the actual configuration of the infrastructure deviates from the intended or documented configuration, causing the infrastructure to fall out of spec.

In contrast, an immutable approach to scaling infrastructure offers a more reliable and secure method. This approach involves creating a similar but separate infrastructure instance, adjusting it as needed, spinning it up when ready, and then decommissioning the old infrastructure once the new one is live.

Some key benefits of immutable infrastructure are as follows:

- The configuration remains consistent and predictable.
- Enhances security by reducing the risk of unintended changes.
- Ensures that each new instance is identical and consistent.
- If issues arise with the new infrastructure, rolling back to the previous version is straightforward.

Summary

In this article, we've learned a lot about the need for infrastructure scaling, its benefits, and the various strategies that can be used.

Let's summarize the learnings in brief:

- Scalable infrastructure provisioning is a critical component of creating infrastructure that is adaptable to fluctuating workloads and user demands.
- Scalable infrastructure provisioning provides several benefits related to availability, scalability, repeatability, and cost-effectiveness.
- Multiple strategies (some overlapping) are available for scaling the infrastructure provisioning and management.
- Containerization is one of the first strategies to make deployments based on containers. Docker is one of the most popular ways to containerize the application.
- Container orchestration becomes a necessity when dealing with multiple containers in an application. This is where container orchestration tools like Kubernetes become important.
- Infrastructure as Code treats infrastructure configuration as code, allowing developers to define the application infrastructure in files that can be versioned, tested, and reused. Popular tools such as Terraform, AWS CloudFormation, and Ansible can be used.
- GitOps leverages a Git workflow combined with continuous integration and continuous delivery (CI/CD) to automate infrastructure updates.
- OPA allows developers to define policies using its scripting language, Rego, which can be used to score resources based on their criticality. This score can be used to control the deployment of infrastructure.
- An immutable approach to scaling infrastructure offers a more reliable and secure method. This approach involves creating a similar but separate infrastructure, adjusting it as needed.



238 Likes • 21 Restacks

Discussion about this post

[Comments](#) [Restacks](#)

Write a comment...



Giuseppe Santoro 🇮🇹 🧑‍💻 Cloud Native Engineer Sep 17, 2024

I wouldn't really put Ansible in the Infrastructure as Code bucket as the image suggests.

Ansible has some plugins to create VMs in Vagrant or Azure but it's not meant for these kind of tasks. It's mostly to run tasks over SSH connecting to VMs already created by tools like terraform.

Furthermore Terraform is not the only IaC tools either. Many other options exist like Crossplane, Pulumi, CDKTF and so on.

♡ LIKE (5) 💬 REPLY



© 2026 ByteByteGo · [Privacy](#) · [Terms](#) · [Collection notice](#)
[Substack](#) is the home for great culture